

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

Localización



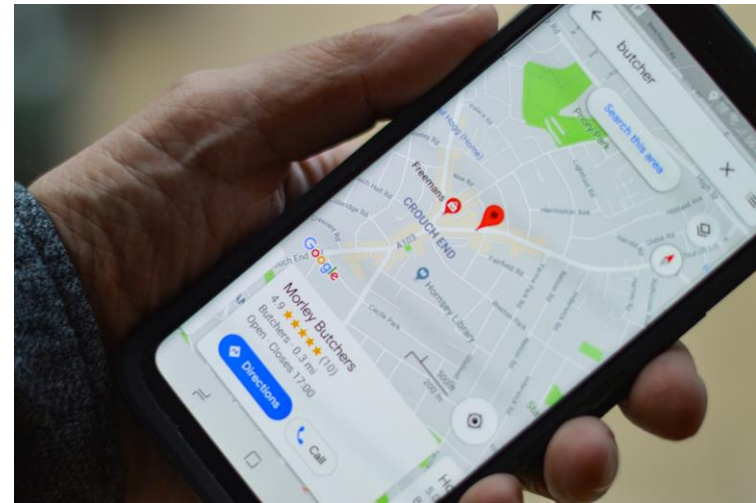
Universidad
Rey Juan Carlos

Localización

Una de las características únicas de las aplicaciones móviles es el conocimiento de la ubicación. Los usuarios de móviles llevan sus dispositivos a todas partes, y añadir el conocimiento de la ubicación a su aplicación ofrece a los usuarios una experiencia más contextual.

Posibles usos de la ubicación:

- Indicaciones en un mapa
- Avisos de peligro
- Restricciones de acceso a contenidos
- Recopilar datos para vendérselos a terceros...



Posición GPS

Tipos de localización

Si tu aplicación contiene una función que comparte o recibe información de ubicación sólo una vez, o durante un periodo de tiempo definido, entonces requiere acceso a la **ubicación en primer plano**. Ejemplos:

- En una aplicación de navegación, para obtener indicaciones giro a giro.
- En una aplicación de mensajería, para compartir la ubicación con otro usuario.

Permisos: ACCESS_COARSE_LOCATION o ACCESS_FINE_LOCATION

Una aplicación necesita acceso a la **ubicación en segundo plano** si una función de la aplicación comparte constantemente la ubicación con otros usuarios o utiliza la *Geofencing API*.

Permiso: ACCESS_BACKGROUND_LOCATION

Dependencias y permisos

Para acceder a la ubicación desde nuestra aplicación debemos configurar las dependencias y los permisos

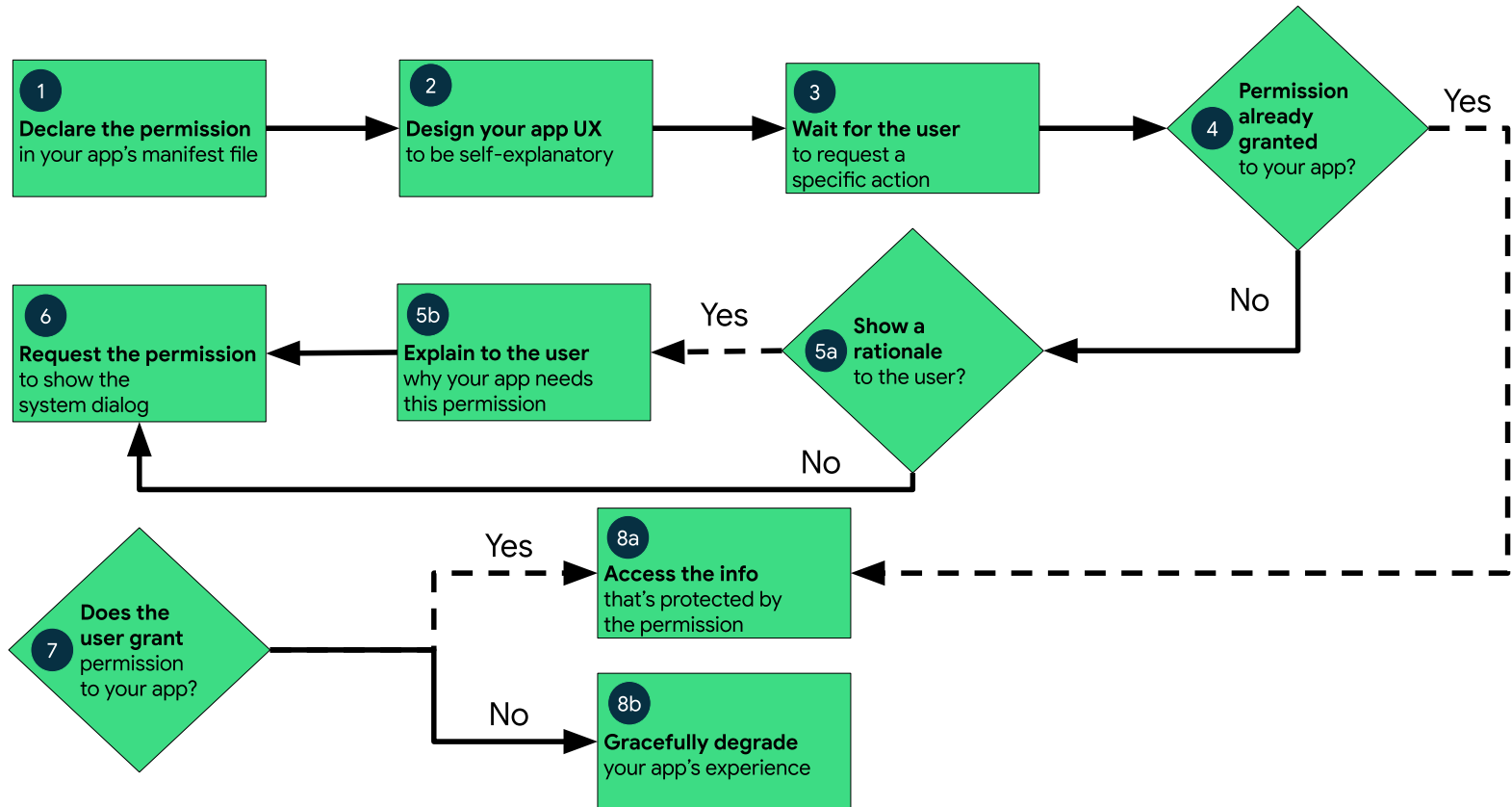
```
dependencies {  
    implementation(libs.play.services.location)  
}
```

build.gradle.kts (:app)

```
<?xml version="1.0" encoding="utf-8"?>  
<manifest xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:tools="http://schemas.android.com/tools">  
  
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />  
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />  
  
    <application  
        ...
```

AndroidManifest.xml

Solicitud de permisos



Comprobar permisos disponibles

Al iniciar la aplicación, **se deben comprobar si los permisos necesarios están disponibles**. Si se ha usado previamente la app, la comprobación será un éxito. Tras la instalación, o si estos han sido revocados, no estarán habilitados.

```
if (ActivityCompat.checkSelfPermission(  
    this,  
    Manifest.permission.ACCESS_FINE_LOCATION  
) == PackageManager.PERMISSION_GRANTED && ActivityCompat.checkSelfPermission(  
    this,  
    Manifest.permission.ACCESS_COARSE_LOCATION  
) == PackageManager.PERMISSION_GRANTED){  
    Log.d("Permisos", "La app tiene los permisos necesarios")  
}else{  
    Log.d("Permisos", "La app no tiene los permisos necesarios")  
}
```

MainActivity.kt

Solicitar permisos al usuario

Si los permisos no están concedidos, hay que solicitarlos y gestionar tanto su aceptación como la negativa del usuario

```
// Configurar la petición de permisos
locationPermissionRequest = registerForActivityResult(
    ActivityResultContracts.RequestMultiplePermissions()
) { permissions ->
    when {
        permissions.getDefault(Manifest.permission.ACCESS_FINE_LOCATION, false) -> {
            Log.d("Permisos", "Permisos aceptados")
        } else -> { // Solo localización aproximada o ningún permiso concedido
            Log.d("Permisos", "Permisos rechazados")
        }
    }
}

// Pedir los permisos
locationPermissionRequest.launch(arrayOf(
    Manifest.permission.ACCESS_FINE_LOCATION,
    Manifest.permission.ACCESS_COARSE_LOCATION))
```

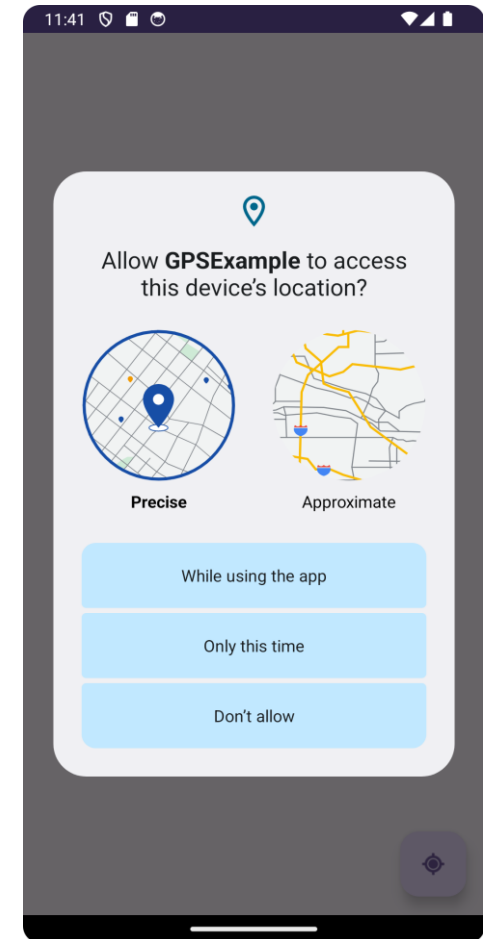
MainActivity.kt

Ejercicio 1

Crea una aplicación que requiera los permisos de localización *coarse* y *fine*.

Al iniciarse la app, comprobará si el usuario ha concedido los permisos necesarios para el funcionamiento de la esta.

1. Si los tiene, mostrará en un campo de texto la palabra "Éxito".
2. Si no los tiene, deberá solicitárselos al usuario:
 - Si el usuario los acepta, se vuelve al punto 1
 - Si los rechaza, se mostrará el mensaje "Fracaso"



Crear un cliente para el LocationProvider

Para **acceder a la localización** del dispositivo hay que **instanciar un cliente de un LocationProvider**. Habitualmente, se utiliza un el proveedor de "fusión", que integra toda la información disponible para ofrecer la ubicación más precisa posible.

```
class MainActivity : AppCompatActivity() {  
    private lateinit var binding: ActivityMainBinding  
    private lateinit var fusedLocationClient: FusedLocationProviderClient  
  
    @SuppressWarnings("MissingPermission")  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        binding = DataBindingUtil setContentView(this, R.layout.activity_main)  
  
        fusedLocationClient =  
        LocationServices.getFusedLocationProviderClient(applicationContext)  
    }  
}
```

MainActivity.kt

Obtener la posición en caché

Una vez el cliente ha sido inicializado, se puede acceder a la posición del móvil.

Hay dos tipos de ubicación disponible, que usaremos dependiendo de las necesidades de la aplicación. Si **no se requiere una localización muy precisa**, podremos utilizar **la última disponible en la caché** del dispositivo y ahorrar así consumo de batería.

```
val posicion = fusedLocationClient.lastLocation.addListener {location ->
    if (location == null) { // GPS apagado
        Toast.makeText(this, "Localización no disponible", Toast.LENGTH_SHORT)
            .show()
    } else {
        Log.d("Location", "%.6f, %.6f".format(location.latitude, location.longitude))
    }
}
```

Obtener la posición actual

Si, por el contrario, **la precisión es relevante en la aplicación a desarrollar**, se puede **forzar a calcular una ubicación precisa y reciente**, en lugar de utilizar la almacenada en caché, que puede estar desfasada.

```
fusedLocationClient.getCurrentLocation(Priority.PRIORITY_HIGH_ACCURACY, object :  
CancellationToken() {  
    override fun onCancelRequested(p0: OnTokenCanceledListener) = CancellationTokenSource().token  
    override fun isCancellationRequested() = false  
})  
.addOnSuccessListener { location: Location? ->  
    if (location == null) { // GPS apagado o petición no resuelta a tiempo  
        Toast.makeText(this, "Localización no disponible", Toast.LENGTH_SHORT).show()  
    } else {  
        Log.d("Location", "%.6f, %.6f".format(location.latitude, location.longitude))  
    }  
}
```

Obtener la posición actual (customizado)

La **configuración por defecto** puede no adecuarse a las necesidades de la app (precisión, velocidad, etc). Podemos **customizar la petición**:

```
val currentLocationRequest = CurrentLocationRequest.Builder()
    .setDurationMillis(1000) // Duración máxima de la petición. Si no, devuelve null
    .setMaxUpdateAgeMillis(2000) // Edad máxima de la posición devuelta
    .setPriority(Priority.PRIORITY_HIGH_ACCURACY) // Relación precisión-consumo
    .setGranularity(Granularity.GRANULARITY_COARSE) // Calidad de la posición obtenida
    .build()
fusedLocationClient.getCurrentLocation(currentLocationRequest, object : CancellationToken() {
    override fun onCancelRequested(p0: OnTokenCanceledListener) = CancellationTokenSource().token
    override fun isCancellationRequested() = false
})
    .addOnSuccessListener { location: Location? ->
        if (location == null) { // GPS apagado o no resuelto a tiempo
            Toast.makeText(this, "Localización no disponible", Toast.LENGTH_SHORT)
                .show()
        } else {
            Log.d("Location", "%.6f, %.6f".format(location.latitude, location.longitude))
        }
    }
}
```

Ejercicio 2

Crea una aplicación que, al pulsar el FAB, muestre por pantalla la posición GPS (latitud y longitud) en la que se encuentra el usuario así como la hora de medición de la localización.

1. Usando la petición por defecto
2. Usando la petición configurada

Su funcionamiento debe comprobarse en el emulador.



Recibir la ubicación periódicamente

Es necesario crear nuevas variables en la actividad para registrar la solicitud de actualizaciones al LocationProvider.

```
import com.google.android.gms.location.LocationCallback
import com.google.android.gms.location.LocationRequest

lateinit var locationRequest: LocationRequest // Requisitos para las actualizaciones
lateinit var locationCallback: LocationCallback // Llamada cuando llega una nueva posición
```

MainActivity.kt

Después, configuramos la solicitud de actualizaciones de ubicación:

```
// Configuración de la solicitud de localización
locationRequest =
    LocationRequest.Builder(1000) // Intervalo al que queremos recibir datos
        .setMinUpdateIntervalMillis(500) // Intervalo más rápido para recibirlos
        .setPriority(Priority.PRIORITY_HIGH_ACCURACY) // Relación precisión-consumo
        .setMinUpdateDistanceMeters(1f) // Distancia entre posiciones para una actualización
        .build()
```

MainActivity.kt

Recibir la ubicación periódicamente (cont.)

Luego, se configura el *callback* a ejecutar cuando una nueva posición de GPS se reciba. Finalmente, se registra la petición.

```
// ... a continuación de lo anterior
// Definición del callback para gestionar la actualización recibida
locationCallback = object : LocationCallback() {
    override fun onLocationResult(locationResult: LocationResult) {
        super.onLocationResult(locationResult)

        // Comprobación extra para ver que nos llega una posición y no viene vacía
        if (locationResult.lastLocation != null){
            val loc = locationResult.lastLocation!!
            Log.d("Location", "%.6f, %.6f".format(loc.latitude, loc.longitude))
        }
    }
}

// Registrar la petición de actualizaciones de localización
fusedLocationClient.requestLocationUpdates(locationRequest, locationCallback, null)
```

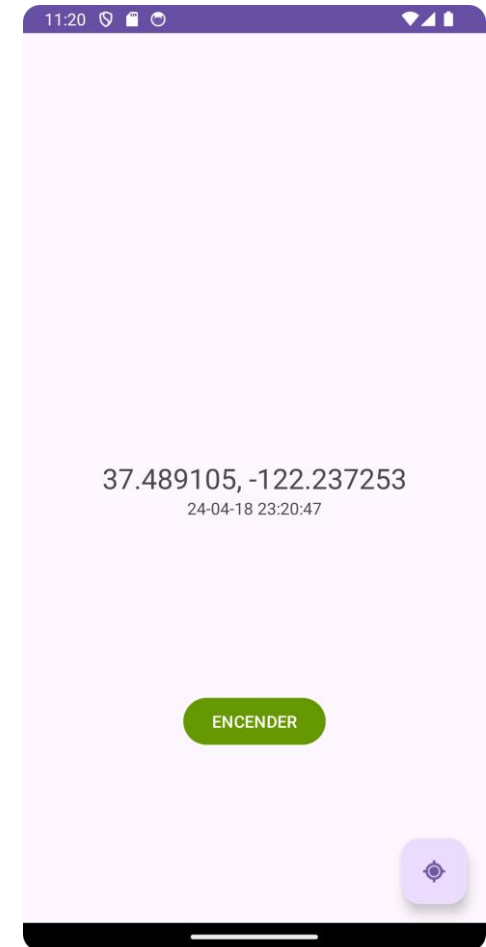
MainActivity.kt

Ejercicio 3

Modifica la aplicación anterior y añade un botón que permita encender y apagar el modo "en directo".

- Cuando la aplicación esté "en directo", la posición mostrada por pantalla se actualiza periódicamente con la llegada de nuevas lecturas de GPS y no únicamente cuando se pulse el FAB.
- Cuando se apague dicho modo, la aplicación funcionará como en el ejercicio anterior.

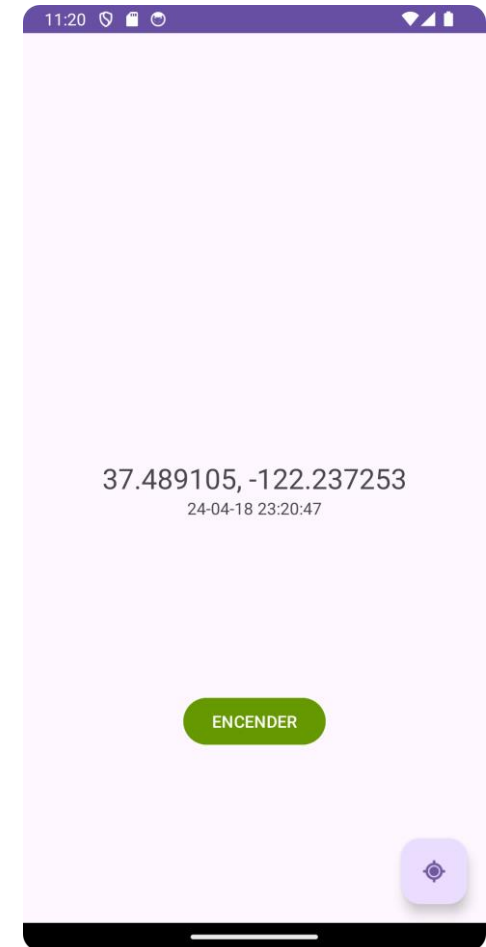
Para parar las actualizaciones de localización sigue el ejemplo de la [documentación oficial](#).



Ejercicio 4 (modo experto)

Partiendo de la aplicación anterior, realiza las modificaciones necesarias para que la ubicación esté guardada en una variable del ViewModel, y la actividad principal observe los cambios sobre la misma.

Para ello, el `FusionLocationProviderClient` y las actualizaciones periódicas o puntuales de la ubicación deberán gestionarse a través del ViewModel, que realizará las operaciones necesarias para obtener la localización y actualizar el objeto *LiveData* correspondiente.



Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>